

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/339325311>

Solving Continuous Optimization Problems by Ant Colony Optimization with Domain Partitioning Technique

Article · May 2018

CITATION

1

READS

436

2 authors, including:



[Jeerayut Wetweeraopong](#)

Khon Kaen University

16 PUBLICATIONS 25 CITATIONS

SEE PROFILE

Solving Continuous Optimization Problems by Ant Colony Optimization with Domain Partitioning Technique

Pirapong Singasathid^{1*} and Jeerayut Wetweerapong²

¹Department of Mathematics, Faculty of Science, Khon Kaen University, pirapongs@kkumail.com

²Department of Mathematics, Faculty of Science, Khon Kaen University, wjeera@kku.ac.th

Abstract

Ant colony optimization (ACO) consists of two basic steps: constructing the candidate solutions by a pheromone model and using the candidate solutions to modify the pheromone values in order to bias future sampling toward high quality solutions. It was initially developed for discrete optimization problems and its extensions for continuous optimization problems have been widely studied in recent years. Many extensions use different techniques which do not strictly conform to the concept of ACO and give relatively low precision solutions. Based on the domain partitioning technique, we present a new continuous ACO, called PACO, which closely follows the concept of ACO and strive to give high precision solutions. This technique iteratively partitions all continuous variable domains to construct and update the pheromone matrix which is used to find a better solution to reduce and repartition the domains. We obtain the suitable parameters for PACO and then compare its performance with those of other previously proposed continuous ACO. The experiments show that it gives better overall results.

Keywords: Ant colony optimization, Continuous optimization

*Corresponding author.

E-mail address: pirapongs@kkumail.com

1 Introduction

The main concept of ant colony optimization (ACO), first introduced by Marco Dorigo in 1992, is based on the foraging behavior of real ants which deposit pheromone on their foraging paths. It has outstanding performance in solving combinatorial optimization problems [4,5,8]. However, to deal with continuous optimization problems, the domain need to be discretized to formulate a pheromone model.

The first attempt is CACO method [1] which discretizes a domain with nest structure, creates random vectors from the nests to find solutions and updates the vectors by better results from ants. Then API method [16] modifies CACO by moving the nests periodically to get close to the successful region. In [9], CIAC method uses the attractive points and allows ants to communicate with other ants to improve searching ability. COAC method [11] divides a domain into many sizes and apply the orthogonal design method to guide ants' movement. These methods get inspiration from discrete ACO but they do not have the pheromone model. So, they are considered as ant related method.

The first algorithm that can be classified as the extension of ACO to continuous functions is ACO_R [6] which uses weighting Gaussian CDF to generate ant based solution and updates pheromone by replacing the worst solution from the archive with the newly generated solution. Then MACACO method [10] uses

multivariate Gaussian PDF and keeps information in the covariance matrix to optimize the search space. $DACO_R$ [12], inspired by ACO_R , enhances ants to explore more regions. HACO [18] combines differential evolution algorithm and continuous probability-based incremental learning to generate better PDF and avoid a local optimum. RACO [3] divides a domain by using the grid method to find an optimal solution not contained in the initial domain for 2-dimensional functions. Although those methods use the pheromone models, they still give relatively low precision solutions.

In this research, we present a new continuous ACO (PACO) by using the domain partitioning technique which iteratively partitions all continuous variable domains to construct and update the pheromone matrix. We sampling candidate solutions from the matrix and use the current best solution to reduce and repartition the domains. The remainder of this paper is organized as follows: Section 2 describes details of PACO and its procedure. Experimental results and analysis are presented in Section 3. Finally, conclusion is given in Section 4.

2 PACO for continuous functions

In general, the ACO approach solves an optimization problem by iterating the following two steps [2]:

- candidate solutions are constructed using a pheromone model;
- the candidate solutions are used to modify the pheromone values in order to bias future sampling toward high quality solutions.

Our proposed PACO consists of these 2 steps by partitioning all continuous variable domains to construct the pheromone matrix and updating the pheromone values. Let $f : [LB_0, UB_0]^{nd} \rightarrow \mathbb{R}$ be an objective function to be minimized where LB_0, UB_0 are the initial lower bound and upper bound for each component of a vector point in the domain. The PACO can be described as follows:

Step 1 : Set the following parameters:

ns : the number for partitioning each variable domain into subintervals

$LB(i), UB(i)$: the current lower and upper bounds for the variable i

na : the number of searching ants for each small group

ng : the number of maximum generations

n_1 : the number of loops for creating the initial pheromone matrix T in phase 1

n_2 : the number of loops for finding the best solution point of the current generation in phase 2

$xbest$: the best solution point at the end of each generation

ρ : the evaporation rate of pheromone where ρ is between 0 and 1

dec : the decreasing rate of domain range

inc : the increasing rate of domain range

δ : the scale for detecting the border zone which contains $xbest$

Step 2 : Partition each domain into ns subintervals.

Let $h(m) = \frac{(UB(m) - LB(m))}{ns}$ be the width of the subinterval m for $m = 1, 2, \dots, nd$. Denote the n th partitioning point of the m th variable by $x(m, n) = LB(i) + nh(i)$ for $n = 1, 2, \dots, ns + 1$. Let T be the $(ns + 1) \times nd$ matrix which represents the amount of pheromone. Initialize $T(i, j) = 1$ for all i, j .

Step 3 : Move each ant randomly to a solution location $(y(1), y(2), \dots, y(nd))$ where each $y(j) = x(j, n(j))$ and $n(j)$ is a uniformly random integer from 0 to ns . Let $xb = (xb(1, nb(1)), \dots, xb(nd, nb(nd)))$ be the best solution obtained among all na ants. Then update T by adding 1 to $T(j, nb(j))$. Repeat this step for n_1 times.

Step 4 : Let T_2 be the matrix obtained by normalizing T as follows:

$$T_2(i, j) = \frac{T(i, j)}{\sum_{i=1}^{ns+1} T(i, j)}$$

Step 5 : Move each ant to a new solution location $(y(1), y(2), \dots, y(nd))$ where each $y(j) = x(j, n(j))$ and $n(j)$ is randomized according to the probabilities $T_2(1, j), \dots, T_2(ns + 1, j)$. Let $xb = (xb(1, nb(1)), \dots, xb(nd, nb(nd)))$ be the best solution obtained among all na ants. Evaporate T_2 by using

$T_2(i, j) = (1 - \rho)T_2(i, j)$ for all i, j . Add 1 only to $T_2(j, nb(j))$ for $j = 1, \dots, nd$. Repeat steps 4 and 5 for n_2 times. At the same time, record the best solution found from this phase 2 and denote it by $xbest = (x(1, nbest(1)), \dots, x(nd, nbest(nd)))$.

Step 6 : Adjust LB and UB by considering the positions $nbest(j)$ of $xbest$.

If $nbest(j) < \delta(ns + 1)$, then we set the new borders as follows:

$LB(j) = \max\{xb(j, nbest(j)) - d, LB_0\}$ where $d = (inc)(UB(j) - LB(j))$

$UB(j) = UB(j)$

If $\delta(ns + 1) \leq nbest(j) \leq (1 - \delta)(ns + 1)$, then we set the new border as follows:

$LB(j) = (dec)LB(j) + ((1 - dec)xb(j, nbest(j)))$

$UB(j) = (dec)UB(j) + ((1 - dec)xb(j, nbest(j)))$

If $nbest(j) > (1 - \delta)(ns + 1)$, then we set the new border as follows:

$LB(j) = LB(j)$

$UB(j) = \min\{xb(j, nbest(j)) + d, UB_0\}$ where $d = (inc)(UB(j) - LB(j))$

Repeat steps 2 to 6 for ng times or until the termination condition is reached. Then output the final $xbest$ and its function value.

3 Experimental results and analysis

First, we find the suitable parameters for PACO. Then, we compare its performance with those of other continuous ACOs.

3.1 Suitable parameters for PACO

The selected configurations of PACO are as follows:

1. The sizes of dimension (nd) are 5, 10, 30.
2. The numbers for partitioning the current domain into subintervals (ns) are 10, 30, 50.
3. The evaporation rates of pheromone (ρ) are 0.05, 0.15, 0.25.
4. The scales for detecting the border zone which contains $xbest$ (δ) are 0.05, 0.1, 0.2.
5. The decreasing rates of domain range (dec) are 0.7, 0.75, 0.8.
6. The number of searching ants for each small group (na) is ns .
7. The number of maximum generations (ng) is 200.
8. The number of loops for creating the initial pheromone matrix T in phase 1 (n_1) is $2ns$.

9. The number of loops for finding the best solution point of the current generation in phase 2 (n_2) is $3ns$.
10. The increasing rate of domain range (inc) is 0.4.

Each function is tested for 20 independent trials of PACO. We consider them as successful runs if their x_{best} 's give the function values less than 10^{-10} . The number of all successful runs (nsc), the average number of function evaluations (nf) and percent of SD (%sd) are recorded.

PACO with each of all 243 combinations of ($nd, ns, \rho, \delta, dec$) is used to minimize 6 selected benchmark functions: unimodal and separable Shifted sphere function; nonseparable Rosenbrock function; multimodal Griewank, Schwefel, Rastrigin, and Ackley functions [6, 10, 18]. The definitions and bounds of all test functions are listed in Table 1.

Table 2 shows the suitable parameters that give the highest success rate and also give the lowest number of function evaluation for each function.

The parameters ns and δ have significant effects on success rates. Parameters $ns = 30, 50$ and $\delta = 0.2$ give the success rate of 100% for almost all cases. Parameters $dec = 0.75$ and 0.8 provide higher success rates while $dec = 0.7$ decreases the domain range too fast and tends to miss the optimum points. Parameters $\rho = 0.15$ is suitable for $ns = 10, 30$ and $\rho = 0.05$ is suitable for $ns = 50$, which indicates that small value of ρ is needed for large number of subintervals. Overall results recommend to use ns large enough with corresponding evaporation rate ρ , $\delta = 0.2$ and $sc = 0.75$ or 0.8 .

3.2 The performance comparisons with other continuous ACOs

After testing all parameter combinations, we use the PACO with (nd, ns, δ, ρ, sc) = (4, 30, 0.05, 0.15, 0.75) for Schwefel function and (4, 10, 0.2, 0.15, 0.75) for the other functions to compare its performance with that of HACO by using the number of function evaluations, basic parameters and results from [18]. The common parameters are also listed in table 3. maxNF is the maximum number of function evaluations allowed and ϵ is the termination condition.

Table 3: Basic parameter settings for low dimension

Parameters	nd	maxNF	ϵ
Sphere	4	170000	0.1
Rosenbrock	4	400000	1.0
Rastrigin	4	900000	0.1
Schwefel	4	900000	0.1
Ackley	4	900000	0.1

Each function is tested for 100 independent trials of PACO. We record the number of function evaluations when x_{best} gives the function values less than ϵ . Table

4 shows the performance comparison between PACO and HACO.

Table 4: The performance comparison between PACO and HACO

Function	HACO		PACO	
	nf	nsc	nf	nsc
Sphere	16764	100	5305	100
Rosenbrock	250407	95	10785	100
Rastrigin	6777	100	6425	100
Ackley	2685	100	9120	100
Schwefel	6777	100	43515	100

From table 4, PACO gives the number of function evaluations less than those of HACO for 3 out of 5 functions. PACO gives the number of function evaluations larger than those of HACO for Schwefel and Ackley functions. However, there is no report of high precision solutions for HACO while PACO with these settings can give the function values less than 10^{-10} .

Next, we investigate the performances for 30 dimensional functions. We use (nd, ns, δ, ρ, sc) = (30, 50, 0.05, 0.05, 0.75) for Schwefel function, (30, 50, 0.2, 0.05, 0.8) for Griewank and Rastrigin functions and (30, 30, 0.2, 0.15, 0.75) for the other functions to compare its performance with those of ACO_R , MACACO and HACO by using the results and basic parameters from [18]. The parameters are also listed in Table 5.

Table 5: Basic parameter settings for high dimension

Parameters	nd	maxNF
Sphere	30	10000000
Rosenbrock	30	60000000
Rastrigin	30	10000000
Griewank	30	10000000
Schwefel	30	10000000

Each function is tested for 30 independent trials of PACO. We record mean and SD of function values obtained from all runs. Table 6 shows the performance comparison between ACO_R , MACACO, HACO and PACO.

From table 6, PACO gives the best performance for unimodal functions since PACO can decrease the domain range to intensify the searching of ants. Compared with ACO_R and HACO, PACO obtains better results for all functions. Moreover, PACO gives the best result for Rosenbrock function. However, MACACO performs slightly better than PACO on multimodal functions. This is probably due to the non-adaptive use of domain adjusting of PACO. Figure 1 shows the convergence performances of PACO for all test functions with $nd = 30$.

Table 1: Test functions to find suitable parameters for PACO

Function	Formula	Bound
Shifted sphere	$f_1(x) = \sum_{i=1}^{nd} (x_i - i)^2$	$[-100, 100]$
Rosenbrock	$f_2(x) = \sum_{i=1}^{nd-1} [100(x_{i+1} - x_i^2)^2 + (x_i - 1)^2]$	$[-100, 100]$
Griewank	$f_3(x) = \frac{1}{4000} \sum_{i=1}^{nd} x_i^2 - \prod_{i=1}^{nd} \cos(\frac{x_i}{\sqrt{i}}) + 1$	$[-600, 600]$
Schwefel	$f_4(x) = 418.98288727243369 \cdot nd - \sum_{i=1}^{nd} x_i \sin(\sqrt{ x_i })$	$[-500, 500]$
Rastrigin	$f_5(x) = \sum_{i=1}^{nd} [x_i^2 - 10 \cos(2\pi x_i) + 10]$	$[-5.12, 5.12]$
Ackley	$f_6(x) = -20 \exp(-0.2(\frac{1}{n} \sum_{i=1}^{nd} x_i^2)^{\frac{1}{2}}) - \exp(\frac{1}{n} \sum_{i=1}^{nd} \cos(2\pi x_i)) + 20 + e$	$[-32.32]$

Table 2: The suitable parameters for PACO

Functions	nd	ns	δ	ρ	dec	nsc	nf	%sd
Sphere	5	10	0.2	0.15	0.75	20	25600	1.02
Rosenbrock		10	0.2	0.15	0.75	20	31800	2.13
Griewank		50	0.2	0.05	0.80	20	799375	0.80
Schwefel		30	0.05	0.15	0.75	20	225450	1.93
Ackley		10	0.2	0.15	0.75	20	44475	0.15
Rastrigin		10	0.2	0.15	0.75	20	32075	0.74
Sphere	10	30	0.2	0.15	0.75	20	290475	1.78
Rosenbrock		30	0.2	0.15	0.75	20	283725	2.21
Griewank		50	0.2	0.05	0.80	19	837500	3.59
Schwefel		30	0.05	0.15	0.75	20	246600	1.74
Ackley		30	0.2	0.15	0.75	20	393300	1.08
Rastrigin		30	0.2	0.15	0.75	19	222395	2.27
Sphere	30	30	0.2	0.15	0.75	20	255825	1.03
Rosenbrock		30	0.2	0.15	0.75	20	336150	1.69
Griewank		50	0.2	0.05	0.75	20	696875	0.80
Schwefel		50	0.05	0.05	0.80	7	743750	1.19
Ackley		30	0.2	0.15	0.75	20	414450	0.48
Rastrigin		50	0.2	0.05	0.80	16	889843	1.05

Table 6: Accuracy comparisons between ACO_R , MACACO, HACO and PACO for nd=30

Function	ACO_R		MACACO		HACO		PACO		
	mean	sd	mean	sd	mean	sd	mean	sd	nsc
Sphere	0	0	0	0	0	0	0	0	30
Rosenbrock	29.93	35.14	7.48	11.68	0.93	1.71	0	0	30
Rastrigin	101.65	21.01	0.00058	0.00009	47.39	9.81	0.0563	0.0504	18
Griewank	0.09	0.18	0	0	0.01	0.03	0.0000247	0.00159	29
Schwefel	3866.227	721.53	0.49	0.1	2911.007	735.74	146.07	0	8

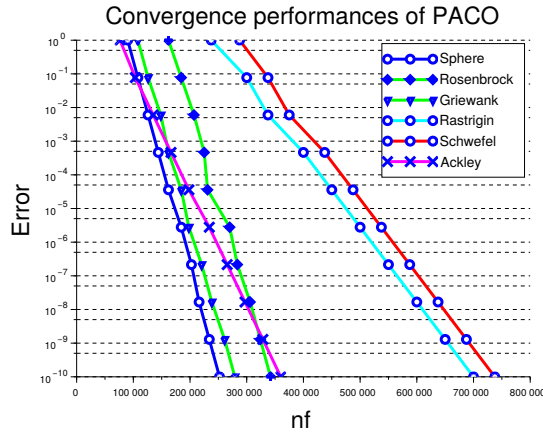


Figure 1: Convergence performances of PACO for $nd = 30$

4 Conclusion

This paper has introduced a competitive and promising PACO for continuous optimization. It only uses a simple domain partitioning technique with the basic concept of ACO. The experiments show that PACO gives better overall results than those of the other continuous ant algorithms on both low dimension and high dimension. However, these results are obtained by using different suitable parameter settings for different types of test function. Future work will enhance the partitioning technique and make parameter settings adaptive for solving various types of test functions.

Acknowledgements

This work was supported in part by development and promotion of science and technology talents project (DPST).

References

- [1] Bilchev G, Parmee IC, The ant colony metaphor for searching continuous design spaces. Proceedings of the AISB workshop on evolutionary computation, university of Sheffield, UK. LNCS. Springer-Verlag. 933. 1995. p. 25–39.
- [2] Blum C. Ant colony optimization: Introduction and recent trends. Physics of Life Reviews. 2005;2(4): 353–373.
- [3] Chen Z, Zhou S, Luo J. A robust ant colony optimization for continuous functions. Expert Systems With Applications . 2017;81:308-320.
- [4] Demirel NÇ, Toksarı MD. Optimization of the quadratic assignment problem using an ant colony algorithm. Applied Mathematics & Computation. 2006;183(1):427–435.
- [5] Dorigo M. Optimization, Learning and Natural algorithms (in Italian). Ph.D. thesis, Dipartimento di Electronica, Politecnico di Milano, Italy.1992.
- [6] Dorigo M, Socha K. Ant colony optimization for continuous domains. European Journal of Operational Research. 2008;185(3): 1155–1173.
- [7] Dorigo M, Stützle T. Ant Colony Optimization. MA, Cambridge:MIT Press; 2004.
- [8] Dorigo M, Gambardella LM. Ant colony system: A cooperative learning approach to the traveling salesman problem. IEEE Transaction on Evolutionary Computation, 1997;1(1):53–66.
- [9] Dréo J, Siarry P. Continuous interacting ant colony algorithm based on dense hierarchy. Future Generation Computer Systems. 2004;20(5): 841–856 .
- [10] Franca F, Coelho GP, Von Zuben FJ, de Faissol Attux RR. Multivariate ant colony optimization in continuous search spaces. Proceedings of the Genetic and Evolutionary Computation Conference (GECCO' 2008), Atlanta, USA. 2008. p. 9–16.
- [11] Hu XM, Zhang J, Li Y. Orthogonal methods based ant colony search for solving continuous optimization problems. Journal of Computer Science and Technology. 2008;23(1):2–18.
- [12] Leguizamón G, Coello CAC. An alternative ACOR algorithm for continuous optimization problems. Proceedings of the seventh international conference on swarm intelligence, ANTS 2010. LNCS. Springer. 6234. 2010. p. 48–59.
- [13] Liao TJ, Montes de Oca MA, Aydın D, Stützle T, Dorigo M. An incremental ant colony algorithm with local search for continuous optimization. Proceedings of the genetic and evolutionary computation conference. GECCO'11. ACM, 2011. p. 125- 132.
- [14] Liao TJ, Montes de Oca MA, Aydın D, Stützle T, Dorigo M. Ant colony optimization for mixed-variable optimization problems. IEEE Transactions on Evolutionary Computation. 2014;18(4): 503–518.
- [15] Liao TJ, Montes de Oca MA, Aydın D, Stützle T, Dorigo M. A unified ant colony optimization algorithm for continuous optimization. European Journal of Operational Research. 2014;234(3): 597–609.
- [16] Monmarché N, Venturini G, Slimane M. On how Pachycondyla apicalis ants suggest a new search algorithm. Future generation computer systems. 2000;16(8):937–946.

- [17] Pourtakdoust SH, Nobahari H. An extension of ant colony system to continuous optimization problems. Proceedings ANTS workshop 2004, LNCS 3172. 2004. p. 294–301.
- [18] Xiao J, Li LP. A hybrid ant colony optimization for continuous domains. Expert Systems with Applications. 2011;38(9): 11072–11077 .
- [19] Yang Q, Chen WN, Yu Z, Gu T, Li Y, Zhang H. Adaptive multimodal continuous ant colony optimization. IEEE Transactions on Evolutionary Computation. 2016;99: 1–14.